

Python Function Complete Task

1. What is a Function?

A Function is a block of code that performs a specific task.

Instead of writing the same code again and again,

we put it inside a function and call it whenever needed.

1) Simple Function :

```
In [4]: def my_name():      # my_name ..is a function name
        print("Azim")
        my_name()          # call the function
```

Azim

```
In [2]: def greet():
        print("Hello Good Evening")
        greet()
```

Hello Good Evening

```
In [3]: def greet():
        print("Hello Good Morning")
        greet()

def greet():
    print("Hello Good Morning")
    greet()

def greet():
    print("Hello Good Morning")
    greet()
```

Hello Good Morning
Hello Good Morning
Hello Good Morning

```
In [4]: # insted of this we can make easier and short code like as below

def greet():
    print("Hello Good Morning")
    greet()
    greet()
    greet()
    greet()
```

```
Hello Good Morning  
Hello Good Morning  
Hello Good Morning  
Hello Good Morning
```

2) Function with No Argument and No Return

```
In [5]: def welcome():  
        print('Welcome to Nit')  
        welcome()
```

```
Welcome to Nit
```

3) Function With One Argument

```
In [11]: def student(name):      # name is argument(1)  
        print("Student name is :", name)  
  
        student("Azim")  
        student("Amir")  
        student("SRK")      # we can add multiple objects
```

```
Student name is : Azim
```

```
Student name is : Amir
```

```
Student name is : SRK
```

4) Function with Two Arguments

```
In [8]: def add(a,b):  
        print("add =", a+b)  
  
        add(10, 20)  
        add(22, 13)
```

```
add = 30
```

```
add = 35
```

```
In [9]: def sub(a,b):  
        print("add =", a-b)      # here function only observe on '-' arithmetic sign on  
  
        sub(10, 20)  
        sub(22, 13)
```

```
add = -10
```

```
add = 9
```

5) Function with Return

```
In [13]: def squre(num):  
         return num*num  
  
         Result= squre(5)  
  
         print(Result)
```

25

```
In [15]: def sub(num):  
         return num-num  
  
         result = sub(5)    # here if we have add 1 attribute then in this call bracet also  
  
         print(result)
```

0

```
In [16]: def add(num):  
         return num+num  
  
         result = add(50)  
  
         print(result)
```

100

```
In [20]: def squre(name):  
         return name*name  
  
         output = squre(2)  
  
         print(output)    # return just focus on sign of arthematic, and give result back to
```

4

```
In [10]: def add(num):  
  
         return num+30*2  
  
         result = add(50)  
  
         print(result)    # return only produce output which is it contain .
```

110

```
In [9]: def num():  
  
         for i in range(10):  
             return i  
  
         print(num())
```

0

```
In [29]: def add(num):  
         return 5
```

```
result = add(50)

print(result)    # it give thus value which is in return function have.
```

5

6) Difference between print and return

```
In [18]: # using print

def add():
    print(10+20)

a = add()    # here return can only give output one time after we print seconde tim
print(a)
```

30

None

```
In [16]: # Using return

def add():

    return 10+20

a = add()

print(a)    # using return it give value which is have.
```

30

7) Multiple Return Values

```
In [22]: def student():
    return 'Azim', 25, 'Solapur'

name,age,city = student()    # call the all function

print(name)
print(age)
print(city)
```

Azim

25

Solapur

8) Positional Arguments

```
In [27]: def emp(name, salary):
    print(salary, name)
```

```
emp("Azim",50000)  # here in print, whatever we put in 1st position it display tho
# In def () 1st name argument mention then in call function emp
```

50000 Azim

```
In [30]: def std(name, ID):

    print(ID)
    print(name)

std("Azim", 101)
std( 44,"Ajim")  # here we add wrong position but in def 1st is name argument the
# then 44 name is print in 2nd position as we mention in print()
```

101
Azim
Ajim
44

```
In [32]: def std(name, age):
    print(age+5)
    print(name*2)

std(" Ajim "*5, 25-2)  # here we also changes in the value in print section
# if in print we add 1 st age then it print 1st age argume
# here we also use this some changes in object variable we
```

28
Ajim Ajim Ajim Ajim Ajim Ajim Ajim Ajim Ajim Ajim

9) Keyword Arguments

```
In [35]: def emp(name,salary):
    print(name)
    print(salary)

emp(salary = 50000, name = "Ajim")  # here we added keyword.
emp(salary= "raj", name = 10000)

# here order does not matter.
# if we add 1st keyword name then he give output name is in 1st position.
# in this 2nd object call functionn we mention salary = raj ,
# then in print section salary is in 2nd position, hence salary = raj in 2nd posi
```

Ajim
50000
10000
raj

```
In [70]: def std(name,age, roll_no):
    print(name,age,roll_no)

std(age = 23, roll_no = 21, name = "Rama")
std(roll_no = 44, name = "Aj", age = 25)  # position is matter..
```

Rama 23 21

Aj 25 44

```
In [19]: def emp(name,salary):
          print(name)
          print(salary-5000)

          emp(salary = 50000+10000, name = "Ajim")    # here we added keyword.
          emp(name = "raj", salary = 10000)

          # here we also use this some changes in print section and also in object variable w
```

Ajim

55000

raj

5000

10) Positional vs Keyword Arguments

```
In [42]: # Positinal
def std(name,age):
    print(name, age)

std(22,"Aj")    # here python consider 22 is name and aj is age, hence as per print(
std("Pk",11)
std(31, "MM")

# keyword
std(age = 22, name = "Aj") # In print section name in 1st position hence name = aj
std(name = "Sk", age = 33) # Now order does not matter only position is important,
std(age = "MM",name = 25)
```

22 Aj

Pk 11

31 MM

Aj 22

Sk 33

25 MM

11) Default Arguments

```
In [80]: def drink(item = "Tea"):    # we make Tea is a default.
          print(item)

          drink()    # if here does not put any value then he auto fill with deafult val
          drink("coffee")    # If we change diff. arg. then it display what we mention.
```

Tea

coffee

```
In [78]: def drink(name, item = "Tea"): # here we make tea as a default, 2 argument use
          print(name,item)
```

```

drink("Azim","coffee") # here we add differant arg. then deafuld item does not co
drink("Lemon_Tea","Aj") # here python consider lemon tea is a name and aj is item,
drink("Tea")             # here we write name is tea then automatically give tea as a
drink("Ajim")            # here we write name then deafuld arg comes after that.

```

Azim coffee
 Lemon_Tea Aj
 Tea Tea
 Ajim Tea

12) Variable Length Arguments(*args)

In [6]: *# without length argument*

```

def total(a,b):
    print(a+b)

```

```

total(10,20) #.. It works because 2 argument, 2 variables
total(10,20,30) # ..error beacause here 3 variable it does not allow

```

30

```

-----
TypeError                                Traceback (most recent call last)
Cell In[6], line 7
      3     print(a+b)
      6 total(10,20) #.. It works because 2 argument, 2 variables
----> 7 total(10,20,30) # ..error beacause here 3 variable it does not allow

TypeError: total() takes 2 positional arguments but 3 were given

```

In [57]:

```

# * = "Collect all positional arguments into one tuple."
def total (*numbers): # Add more amount of numbers.
    print(sum(numbers)) # here we use only sum keyword (numbers is tuple)and nothi

total(10,30) # python add all element.
total(10,30,40)
total(10,20,30,40,)

```

40

80

100

In [9]: **def** total (*nums): *# * is important not name .*

```

    print(sum(nums))
    print(type(total)) # type is function
    print(type(nums))  # type is tuple.

```

```

total(10,30)
total(10,30+40)
total(10,20,30*40,)
total(50-40,300*2,100/25,10%100) # here it calculate all then finally take sum of

```

```

40
<class 'function'>
<class 'tuple'>
80
<class 'function'>
<class 'tuple'>
1230
<class 'function'>
<class 'tuple'>
624.0
<class 'function'>
<class 'tuple'>

```

13) Loop Through *args

```

In [46]: def display(*items):

          for item in items:
              print(item)

          display("Pen", "Book", "Laptop")

```

```

Pen
Book
Laptop

```

```

In [49]: def display(*details):

          for item in details:
              print(item*2)

          display("Water bottel", "School Bag", "Laptop")

```

```

Water bottelWater bottel
School BagSchool Bag
LaptopLaptop

```

Add Numbers Without sum()

```

In [55]: def total(*args):
          result = 5 #...5 = num
          count = 2

          for num in args:
              result = result + num
              count += 1

          print(result)
          print(count)

          total(10, 20, 30, 40) # it display total result of this object then add 5 as we me

```


105
6

14) Keyword Variable Argument(**kwargs)

```
In [2]: def student(**kwargs):    # store name = value pairs.
        print(kwargs)
        print(type(kwargs))    # type is dict.

        student(name="Azim", age = 22)
```

```
{'name': 'Azim', 'age': 22}
<class 'dict'>
```

```
In [17]: def employee(**details):

        print(details)
        print(type(details))

        employee(emp_name = "Rajesh",
                  emp_id = 101,
                  emp_salary = 40000,
                  emp_city = "Hyd")
```

```
{'emp_name': 'Rajesh', 'emp_id': 101, 'emp_salary': 40000, 'emp_city': 'Hyd'}
<class 'dict'>
```

15) Local Variable

```
In [22]: def data():
        a = 1000    # here a value is written after def function
        print(a)

        data()    #data() takes 0 positional arguments but 1 was given
```

1000

16) Global Variable

```
In [23]: a = 500    # here a value is written before def function
        def data():
            print(a)

        data()
```

500

17) Lambda Add

```
In [29]: # 1st Normal function
def squre(x):

    return x * x

print(squre(6))
```

36

```
In [37]: # By using Lambdawe get same output with Less code.
add = lambda a,b : a+b

print(add(10,5))
```

15

Lamba with one Argument

```
In [39]: cube = lambda x: x**3

print(cube(2))
```

8

Lambda with Multiple Arguments

```
In [42]: average = lambda a, b, c : (a+b+c)/3

print(average(11,22,33))
```

22.0

Lambda with If-Else

```
In [46]: largest = lambda a,b : a if a>b else b

print(largest(40,10))
```

40

Lambda with map()

```
In [49]: # 1st Without Lambda:

nums = [1,2,3,4]

result = []

for n in nums:
    result.append(n*n)
```

```
print(result)
```

```
[1, 4, 9, 16]
```

```
In [50]: # by using lambda:
```

```
nums = [1,2,3,4]

result = list(map(lambda x: x*x, nums))

print(result)
```

```
[1, 4, 9, 16]
```

Lambda with filter()

```
In [52]: nums = [1,2,3,4,5,6,7,8,9,10]
```

```
even = list(filter(lambda x : x%2 == 0, nums)) # here %2== 0 means reminder need
# when we devide any value with 2,

print(even)
```

```
[2, 4, 6, 8, 10]
```

Lambda with sorted()

```
In [58]: students = [
    ('Ajay',80),
    ('Ali',75),
    ('Amir',90)
]
result = sorted(
    students, # Lambda is Used only once.
    key = lambda x :x[1] # here 1 means is select 1 position value in 1st.(0=ajay)
)

print(result)
# Use the second value (marks) for sorting.
```

```
[('Ali', 75), ('Ajay', 80), ('Amir', 90)]
```

18) Recursive Function

```
In [ ]: # It is a function that calls itself.
# A function repeatedly calls itself until a stopping condition is reached.
```

```
In [73]: def countdown(n):
```

```
    if n == 0:
        return 0 # here 0 is a stopping point hence return also be 0 or
# we write only single return also.
```

```

print(n)

countdown(n-1)

countdown(5)
# here in 1st,2,3,4 call is 5,4,3,2 hence condition is false is not = 0
# but in 5th call condition is satisfied is = 0 hence is true, then it will stop here

```

5
4
3
2
1

19) Using factorial recursive function:

In [72]: *# using factorial recursive function:*

```

def factorial(n):

    if n == 1:      # Think return 1 is like wall, we need to stop here.
        return 1   # 1! = 1, So when recursion reaches 1, we already know the answer
                   # "Stop recursion. The factorial of 1 is 1."

    return n * factorial(n-2) # here return 1 means it multiply the result by 1 at
                             # because our condition is satisfied n==1.

print(factorial(5))

```

15

20) task on student Result function

In [78]:

```

def result (marks):
    if marks >= 35:
        return "Pass"
    return "Fail"

student_marks = int(input("Enter marks : "))

print(result(student_marks))

```

Pass

21) task on Employee details :

In [84]:

```

def employee(salary):
    if salary <= 30000:
        return "Freshers Employee"
    return "Senior Employee"

```

```
employee_salary = int(input("Enter salary : "))  
  
print(employee(employee_salary))
```

Freshers Employee

In []:

In []:

In []:

In []:

In []:

In []:

In []: